



IT582: Foundations of Machine Learning

Lab-7: Naive Bayes, Perceptron and Logistic Regression

Course Instructor: Srimanta Mandal

Submission Instructions

Upload **only one Jupyter Notebook (.ipynb)** file to Moodle.

The notebook should contain all code, outputs, plots, observations and conclusions.

Name the notebook as:

`[studentid]_lab7.ipynb`

Implement Naive Bayes, Perceptron, Logistic Regression and Gradient Descent with Momentum from scratch. Do not use built-in classifiers or optimizers.

Dataset-loading, train-test splitting and evaluation utilities may be used.

Task 1: Naive Bayes Classification on Titanic Data

Use the **Titanic passenger dataset** available from OpenML (`data_id=40945`).

Use the following categorical attributes:

`pclass, sex, embarked`

to predict

`survived.`

Remove observations having missing values in the selected attributes and use a stratified 70%-30% train-test split.

Implement a categorical Naive Bayes classifier from scratch. Estimate the class priors and conditional probabilities from the training data. Use Laplace smoothing while estimating the conditional probabilities.

For prediction, use the log-score

$$S(c) = \log P(Y = c) + \sum_{j=1}^d \log P(X_j = x_j \mid Y = c)$$

and assign the class having the largest score.

Perform the following:

- Report the estimated class priors.
- Display the conditional probability tables for the selected attributes.
- Predict all test observations and report test accuracy and confusion matrix.
- Select any five test passengers and display their attributes, true class, class log-scores and predicted class.

Write observations.

Task 2: Classification of Linearly Separable Data

Use the **Iris dataset** available through

```
sklearn.datasets.load_iris()
```

Retain only

Iris Setosa and Iris Versicolor

and use the two features

$x_1 = \text{Petal Length}, \quad x_2 = \text{Petal Width}.$

Plot the two classes before training the classifiers.

Use a stratified 70%–30% train–test split and standardize the features using the training-set statistics only.

A. Perceptron

Use labels $y \in \{-1, +1\}$ and implement the Perceptron from scratch.
For a misclassified observation, use

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha y^{(i)} \mathbf{x}^{(i)}.$$

Train for a maximum of 100 epochs or stop earlier if an entire epoch contains no misclassified observations.

Report:

- Number of misclassified training samples in each epoch.

- Number of epochs required for convergence.
- Training and test accuracy.
- Final decision boundary.

B. Logistic Regression

Convert the labels to $\{0, 1\}$ and implement Logistic Regression from scratch using

$$h_{\theta}(\mathbf{x}) = \frac{1}{1 + \exp(-\theta^T \mathbf{x})}.$$

Train the model using Batch Gradient Descent and the logistic negative log-likelihood discussed in class.

Use 0.5 as the classification threshold.

Plot:

- Loss versus iteration.
- Final decision boundary together with the training data.

Report training and test accuracy.

Compare the behaviour of Perceptron and Logistic Regression on this dataset.

Task 3: Classification of Non-Linearly Separable Data

Again use the Iris dataset, but retain only

Iris Versicolor and Iris Virginica.

Use the same two features:

$$x_1 = \text{Petal Length}, \quad x_2 = \text{Petal Width}.$$

Plot the data first and compare its class overlap with the dataset used in Task 2.

Use a stratified 70%–30% train–test split.

A. Linear Classifiers

Using the original features, train:

- Perceptron
- Logistic Regression

using the same implementations from Task 2.

For both classifiers:

- Report training and test accuracy.
- Plot the decision boundary.

For the Perceptron, also plot the number of misclassified training samples versus epoch and check whether it reaches zero.

B. Nonlinear Feature Expansion

For Logistic Regression, replace the original feature representation with

$$\phi(\mathbf{x}) = [1 \quad x_1 \quad x_2 \quad x_1x_2 \quad x_1^2 \quad x_2^2]^T.$$

Retrain Logistic Regression using the transformed features.

Plot the resulting decision boundary in the original (x_1, x_2) plane and compare it with the linear Logistic Regression boundary.

Report the test accuracies before and after feature expansion.

Write observations on:

- Difference between the class separation in Tasks 2 and 3.
- Behaviour of the Perceptron when the training data are not linearly separable.
- Whether nonlinear feature expansion improves Logistic Regression.

Task 4: Gradient Descent with Momentum

Use the **linear Logistic Regression problem from Task 3**.

Initialize

$$\boldsymbol{\theta}_0 = \mathbf{0}$$

and use the same training data, initialization and learning rate for both optimization methods.

First train using ordinary Batch Gradient Descent.

Then implement Gradient Descent with Momentum using

$$\mathbf{v}_0 = \mathbf{0},$$

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t - \eta \nabla J(\boldsymbol{\theta}_t),$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \mathbf{v}_{t+1}.$$

Use

$$\eta = 0.05, \quad \beta \in \{0, 0.9\}, \quad 200 \text{ iterations.}$$

Plot the loss versus iteration for both values of β on the same figure.

Report:

- Loss after iterations 10, 50, 100 and 200.
- Final training and test accuracy.
- Difference in convergence behaviour between ordinary Gradient Descent and Gradient Descent with Momentum.

Write a short conclusion explaining the roles of $\mathbf{v}_t$, β and η .